Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Layout (1)

Outline

- Widget
- Container
- Row & Column

Widget

- Widgets describe what **their view should look like given their current configuration and state**, that is inspired from React
- If a widget's state changes, the widget rebuilds its description, which the framework diffs against the previous description in order to determine the minimal changes needed in the underlying render tree to transition from one state to the next

Common Basic Widgets

- Text
- Row, Column
- Container

A Simple Hello World

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Hello, world!',
        textDirection: TextDirection.ltr,
      ),
    ),
  );
```

Hello World Output

```
Hello, world!
```

Container

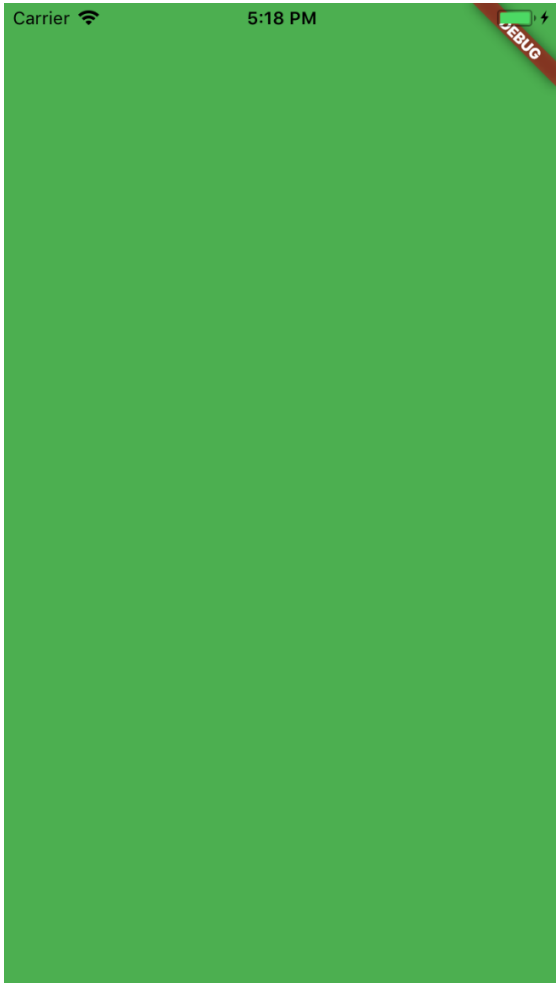
- A convenience widget that combines common painting, positioning, and sizing widgets.
- A container first surrounds the child with padding (inflated by any borders present in the decoration) and then applies additional constraints to the padded extent (incorporating the width and height as constraints, if either is non-null). The container is then surrounded by additional empty space described from the margin.

Container Class

- The Container widget is used to contain a child widget with the ability to apply some styling properties.
- If the Container widget has no child it will automatically fill the given area on the screen, otherwise it will wrap the height & width of the given child element.

Color Property

```
Center(
  child: Container(
    color: Colors.green,
  ),
);
```



Colors Class

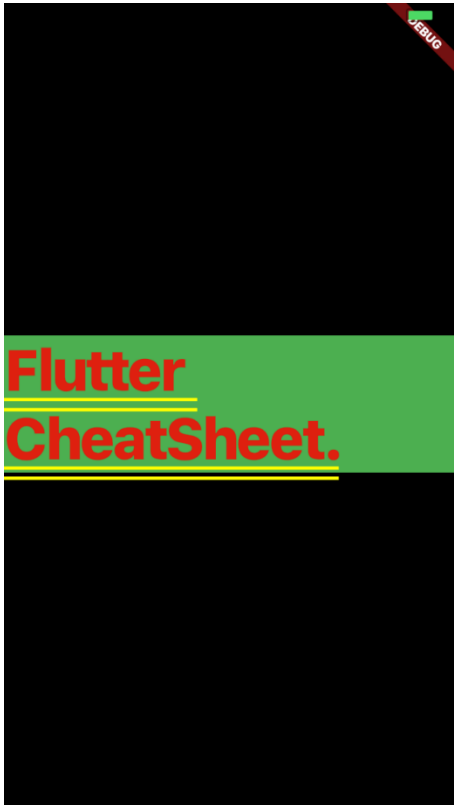
- Use Colors Class with the color name:
  - color: Colors.*green*
- Use Colors Class with added shade:
  - color: Colors.*green*[400]
  - OR
  - color: Colors.*green*.**shade400**
- Use Colors Class with hexa:
  - Color(0xFFFFFF)
  - Color.fromARGB(0xFF, 0x42, 0xA5, 0xF5);

Container - Child Property

- Provide a child widget to be contained by the container, the container will wrap the width & height of this child
- This widget can only have one child. To lay out multiple children, let this widget's child be a widget such as Row, Column, or Stack, which have a children property, and then provide the children to that widget (*will be discussed later*)

Child Property

```
Center(
  child: Container(
    color: Colors.green,
    child: Text("Flutter CheatSheet."),
  ),
);
```

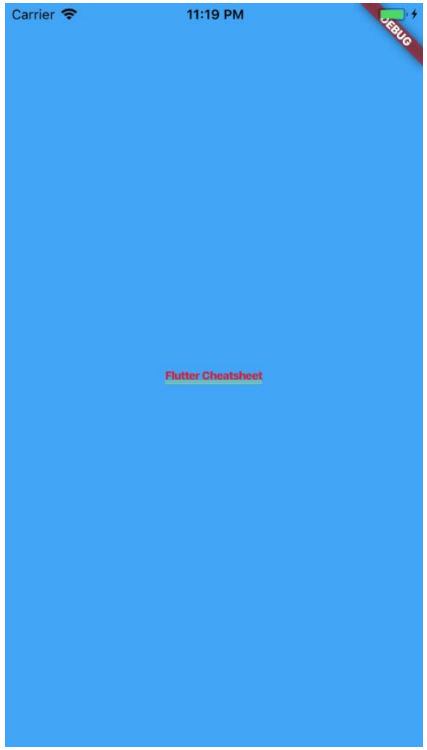


Container - Alignment Property

- Alignment property is applied to align the child widgets
- Alignment take 2 parameters x and y
  - Alignment(0.0, 0.0) represents the center of the rectangle

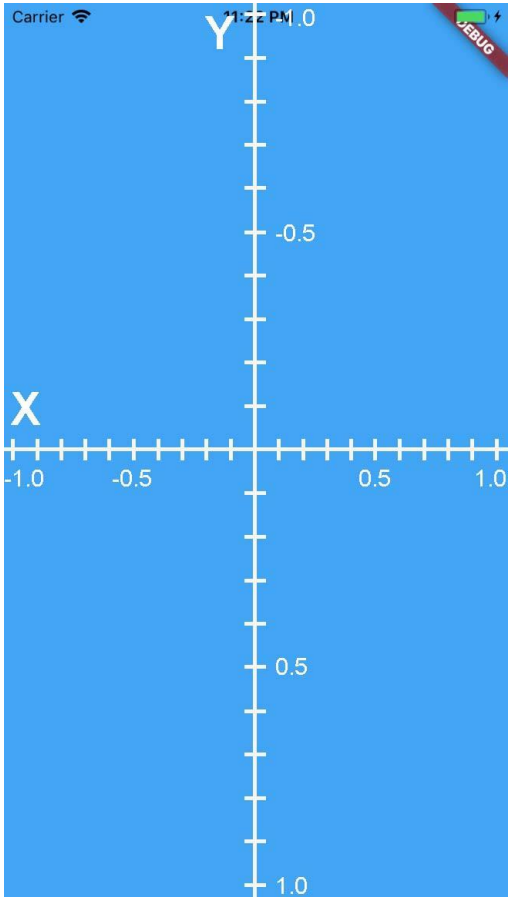
Child Property

```
Center(
  child: Container(
    color: Color.fromARGB(255, 66, 165, 245),
    child: new Text("Flutter Cheatsheet",
      style: TextStyle(
        fontSize: 10.0
      ),
    ),
    alignment: Alignment(0.0, 0.0),
  ),
);
```



Alignment X and Y

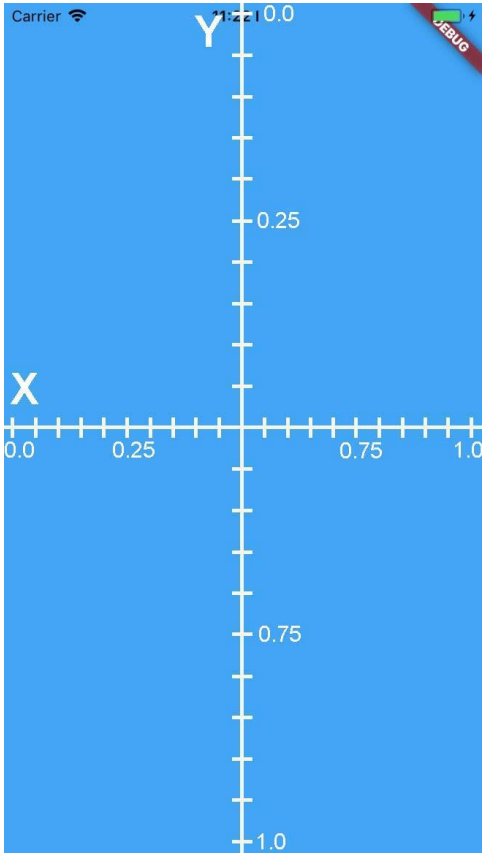
- Constants
  - Alignment.bottomCenter **OR** Alignment(0.0, 1.0)
  - Alignment.bottomLeft **OR** Alignment(-1.0, 1.0)
  - etc.



FractionalOffset

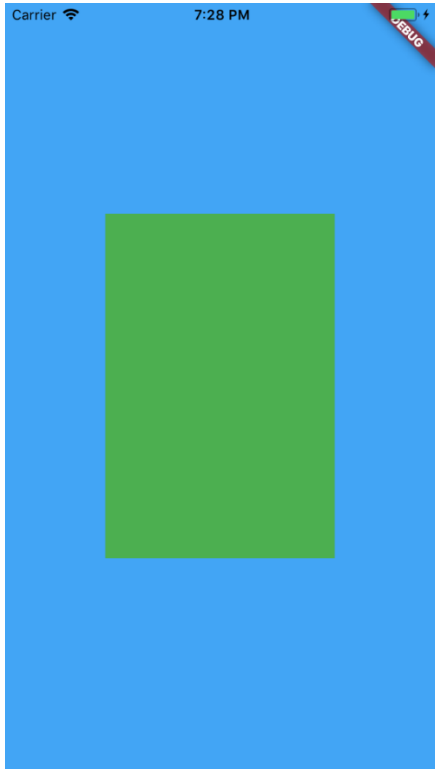
- FractionalOffset and Alignment are two different representations of the same information: the location within a rectangle relative to the size of the rectangle. The difference between the two classes is in the coordinate system they use to represent the location.
- FractionalOffset uses a coordinate system with an origin in the top-left corner of the rectangle whereas Alignment uses a coordinate system with an origin in the center of the rectangle.

FractionalOffset Coordinate



Container - Constraints Property

```
Center(
  child: Container(
    color: Color.fromARGB(255, 66, 165, 245),
    alignment: AlignmentDirectional(0.0, 0.0),
    child: Container(
      color: Colors.green,
      constraints: BoxConstraints(
          maxHeight: 300.0,
          maxWidth: 200.0,
          minWidth: 150.0,
          minHeight: 150.0
      ),
    ),
  ),
);
```

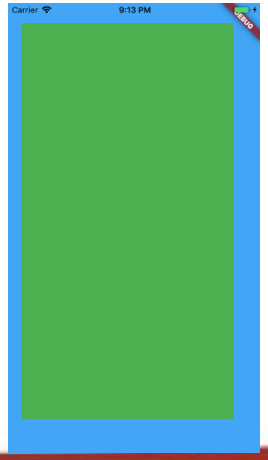


Container - Margin Property

- A margin is just a property specifying to add empty space to surround of the container using an EdgeInsets constant value.
  - **EdgeInsets.all**(20.0)
  - **EdgeInsets.symmetric**(vertical: 20.0, horizontal: 50.0)
  - **EdgeInsets.fromLTRB**(20.0, 30.0, 40.0, 50.0)
  - **EdgeInsets.only**(
    left: 20.0,
    bottom: 40.0,
    top: 50.0
)

Container - Constraints Property

```
Center(
  child: Container(
    color: Color.fromARGB(255, 66, 165, 245),
    alignment: AlignmentDirectional(0.0, 0.0),
    child: Container(
      color: Colors.green,
      margin: new EdgeInsets.fromLTRB(20.0, 30.0, 40.0, 50.0),
    ),
  ),
);
```



Container - Padding Property

- A padding is just a property specifying to add empty space to inscribe inside the container using an EdgeInsets constant value same as the margin.

Container - Constraints Property

```
Center(
  child: Container(
    color: Color.fromARGB(255, 66, 165, 245),
    alignment: AlignmentDirectional(0.0, 0.0),
    child: Container(
      margin: new EdgeInsets.only(
          left: 20.0,
          bottom: 40.0,
          top: 50.0
      ),
      padding: new EdgeInsets.all(40.0),
      color: Colors.green,
      child: Text("Flutter Cheatsheet"),
      constraints: BoxConstraints.expand(),
    ),
  ),
);
```

Container - Decoration Property

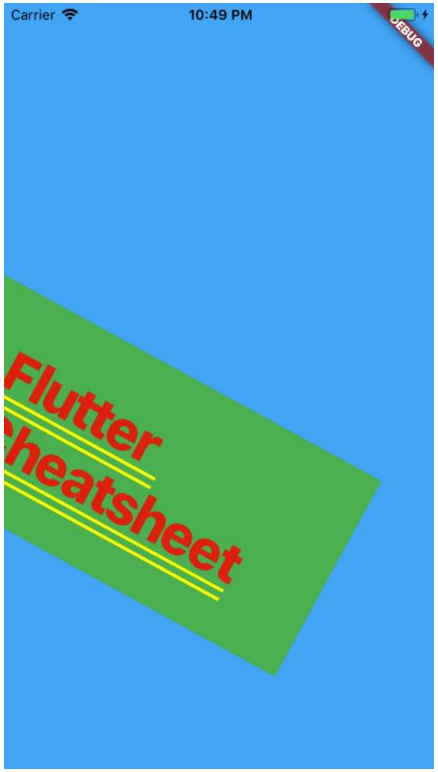
- A decoration property can be applied behind the given container.
- The value can be:
  - BoxDecoration Class
  - FlutterLogoDecoration Class
  - ShapeDecoration Class
  - UnderlineTabIndicator Class

Container - ForegroundDecoration

- A decoration property can be applied in front of the given container.
- The value options are the same Decoration

Container - Transform Property

```
Center(
  child: Container(
    color: Color.fromARGB(255, 66, 165, 245),
    alignment: AlignmentDirectional(0.0, 0.0),
    child: Container(
      padding: new EdgeInsets.all(40.0),
      color: Colors.green,
      child: Text("Flutter Cheatsheet"),
      transform: new Matrix4.rotationZ(0.5)
    ),
  ),
);
```



Row Class

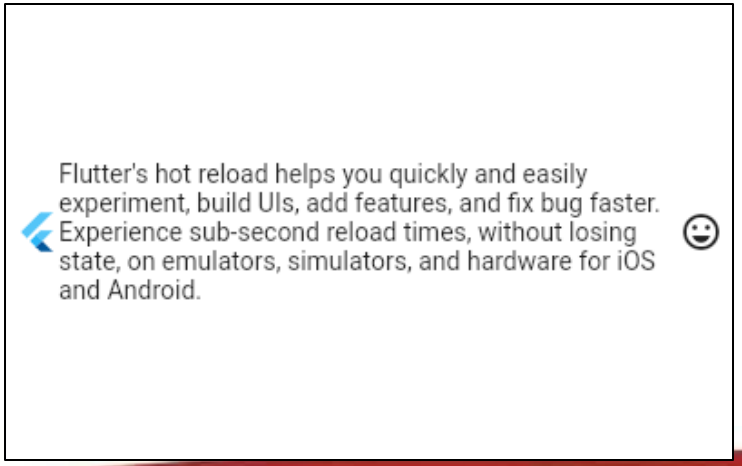
- A widget that displays its children in a horizontal array.
- To cause a child to expand to fill the available horizontal space, wrap the child in an Expanded widget.
- The Row widget does not scroll (and in general it is considered an error to have more children in a Row than will fit in the available room).

Row Example

```
Row(
  children: [
    Container(
      color: Colors.orange,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
    Container(
      color: Colors.blue,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
    Container(
      color: Colors.purple,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
  ],
)
```

Row with Expanded

```
Row(
  children: <Widget>[
    const FlutterLogo(),
    const Expanded(
      child: Text("Flutter's hot reload helps you quickly and easily experiment, build UIs, add features, and fix bug faster. Experience sub-second reload times, without losing state, on emulators, simulators, and hardware for iOS and Android."),
    ),
    const Icon(Icons.sentiment_very_satisfied),
  ],
)
```

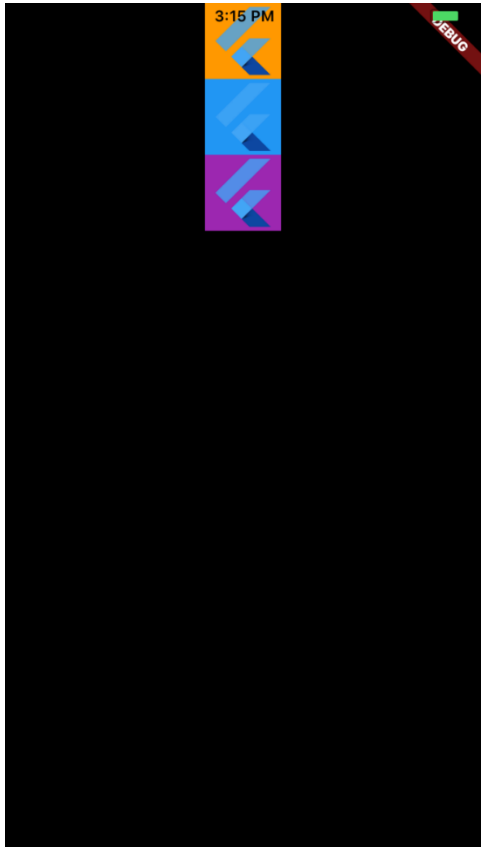


Column Class

- A widget that displays its children in a vertical array.
- To cause a child to expand to fill the available vertical space, wrap the child in an Expanded widget.
- The Column widget does not scroll (and in general it is considered an error to have more children in a Column than will fit in the available room).

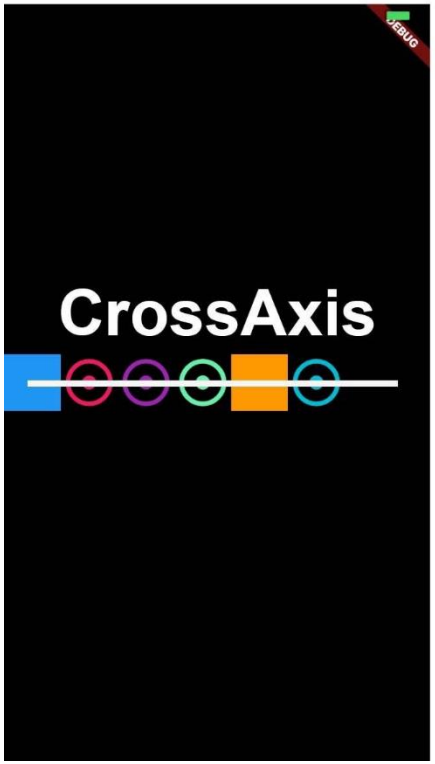
Column Example

```
Column(
  children: [
    Container(
      color: Colors.orange,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
    Container(
      color: Colors.blue,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
    Container(
      color: Colors.purple,
      child: FlutterLogo(
        size: 60.0,
      ),
    ),
  ],
)
```

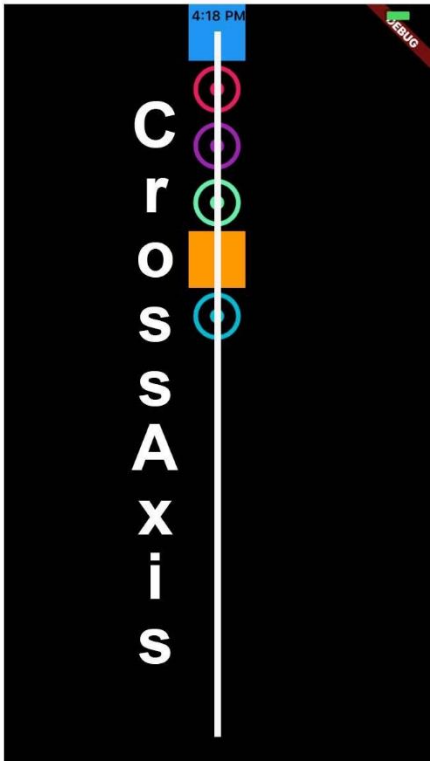


CrossAxis in Row and Column

Column



Row



CrossAxisAlignment Property

- We can use the ***crossAxisAlignment*** property to align our child widget in the desired direction, for example:
  - crossAxisAlignment.**start** would place the children with their start edge aligned with the start side of the cross axis
  - CrossAxisAlignment.**center** would place the children so that their centers align with the middle of the cross axis.
  - Others:
    - **CrossAxisAlignment.end**
    - **CrossAxisAlignment.stretch**
    - **CrossAxisAlignment.baseline**

TextDirection Property

- Determines the order to lay children out horizontally and how to interpret start and end in the horizontal direction.
  - textDirection: TextDirection.**rtl**
  - textDirection: TextDirection.**ltr**

VerticalDirection Property

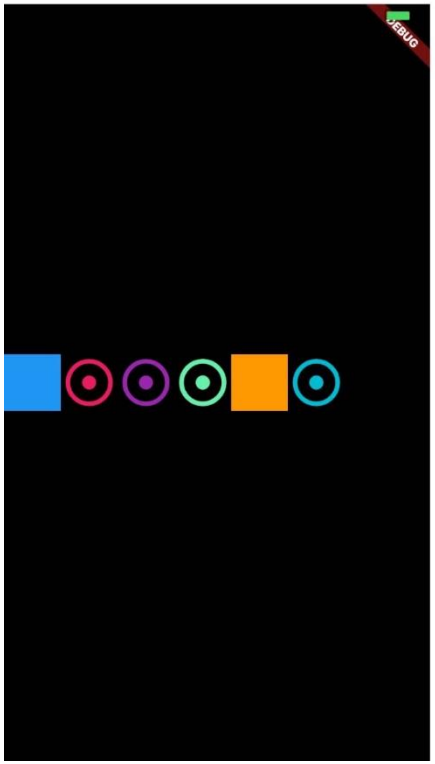
- Determines the order to lay children out vertically and how to interpret start and end in the vertical direction.
  - VerticalDirection.**down**
  - VerticalDirection.**up**

MainAxisAlignment Property

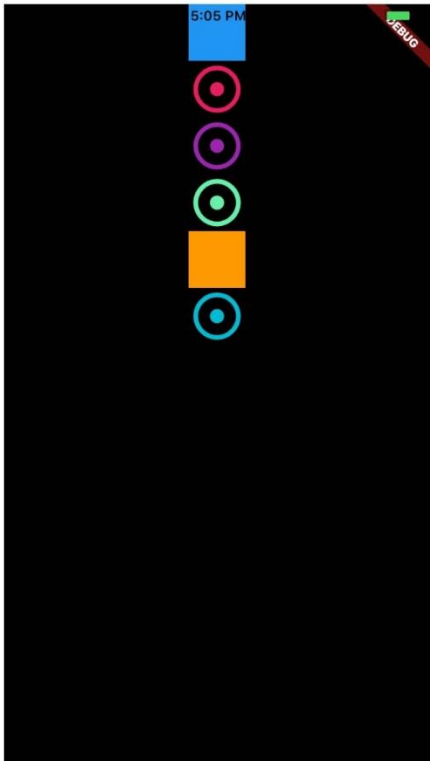
- The positioning of the child widgets on the main axis.
  - MainAxisAlignment.start
  - MainAxisAlignment.center
  - MainAxisAlignment.end
  - MainAxisAlignment.spaceAround
  - MainAxisAlignment.spaceBetween
  - MainAxisAlignment.spaceEvenly

MainAxisAlignment.start

Row

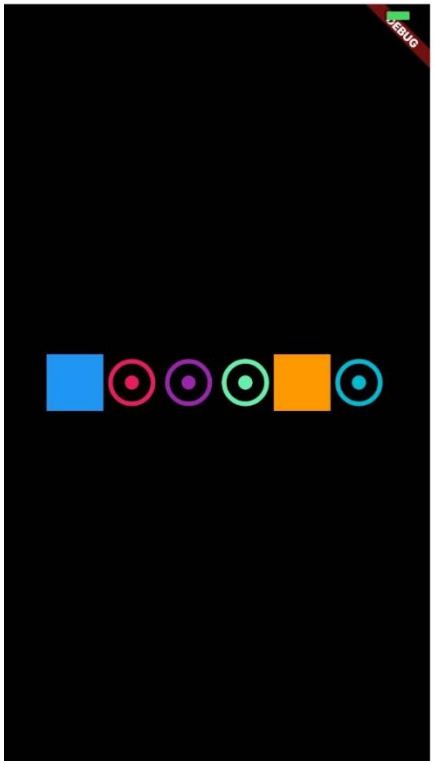


Column

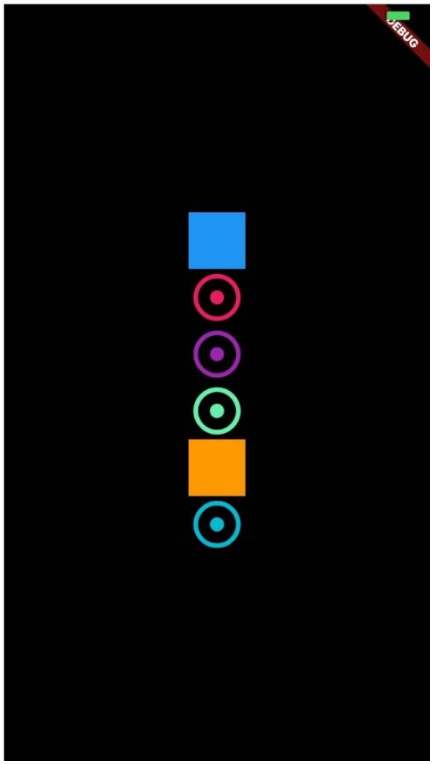


MainAxisAlignment.center

Row

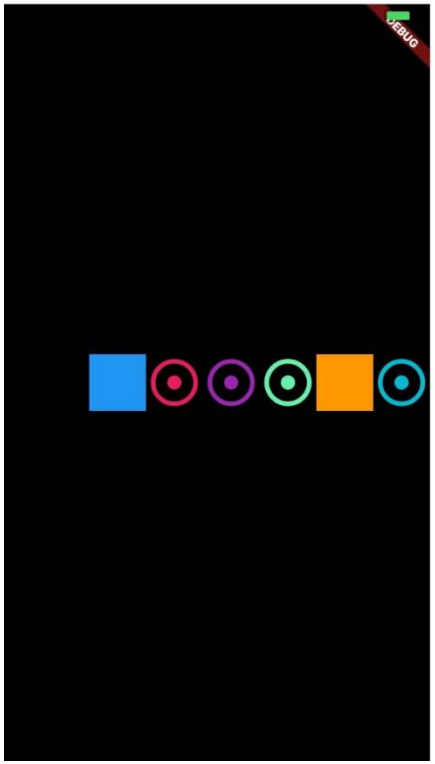


Column

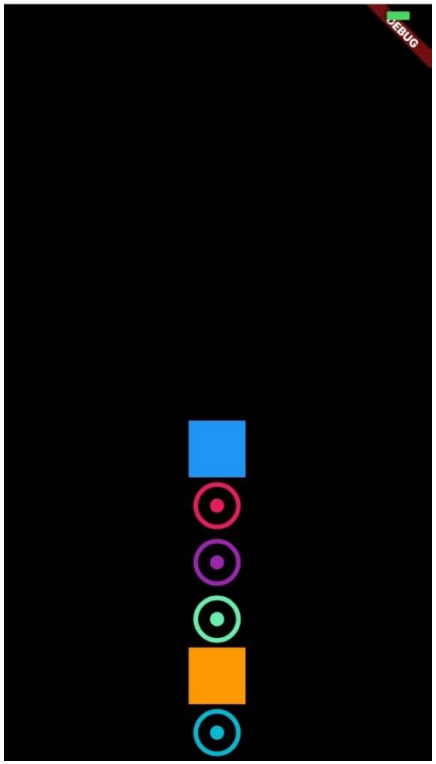


MainAxisAlignment.end

Row

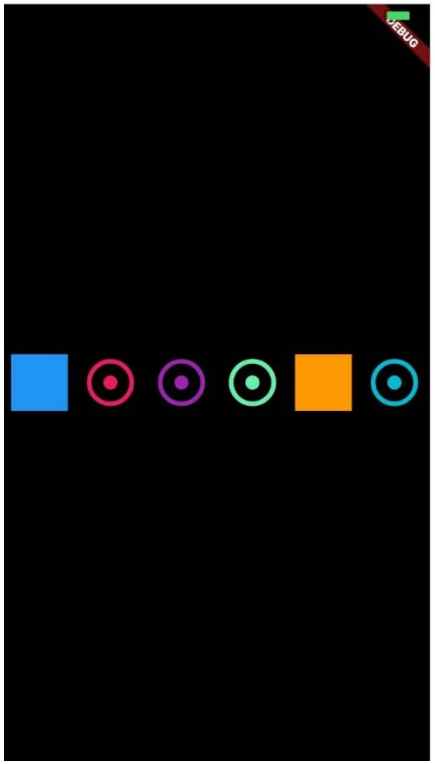


Column

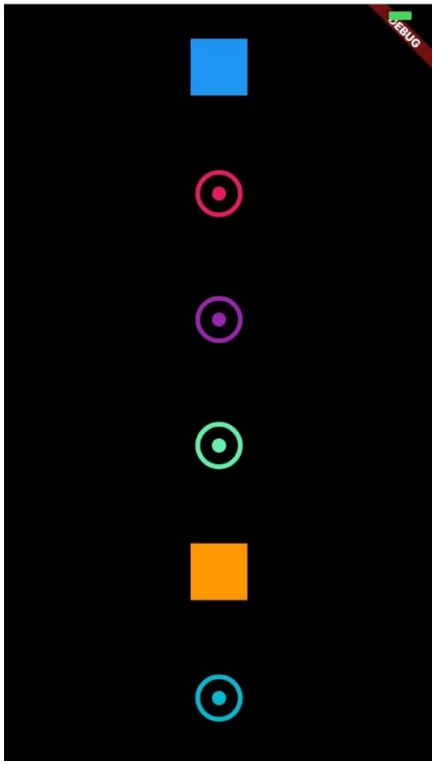


MainAxisAlignment.spaceAround

Row

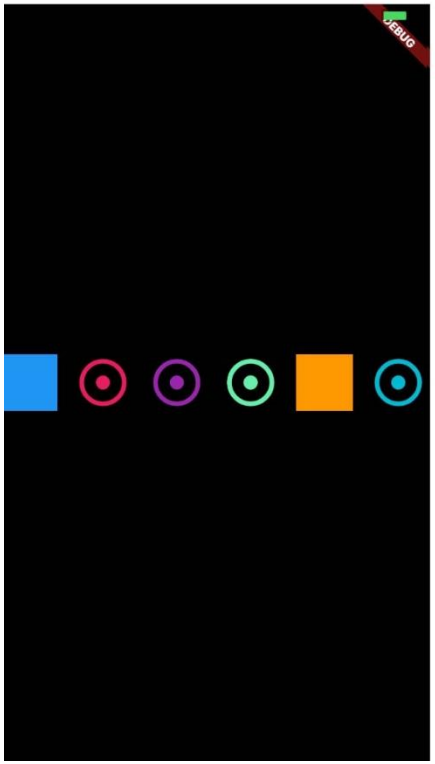


Column

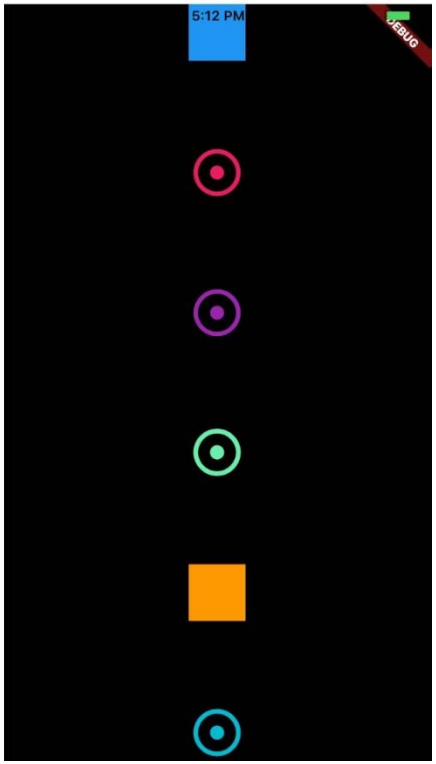


MainAxisAlignment.spaceBetween

Row

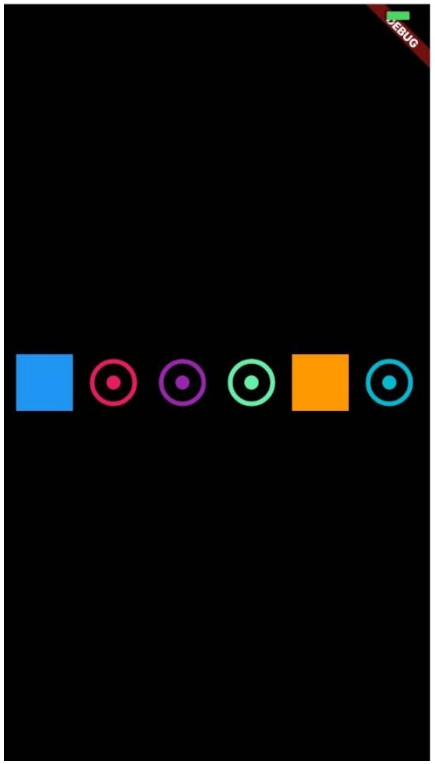


Column

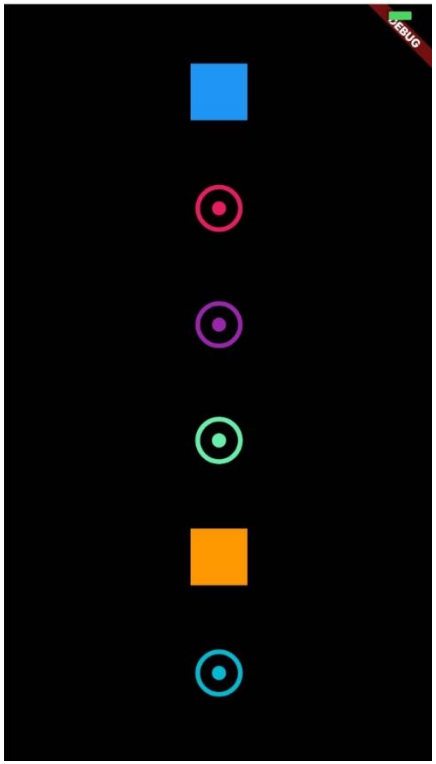


MainAxisAlignment.spaceEvenly

Row



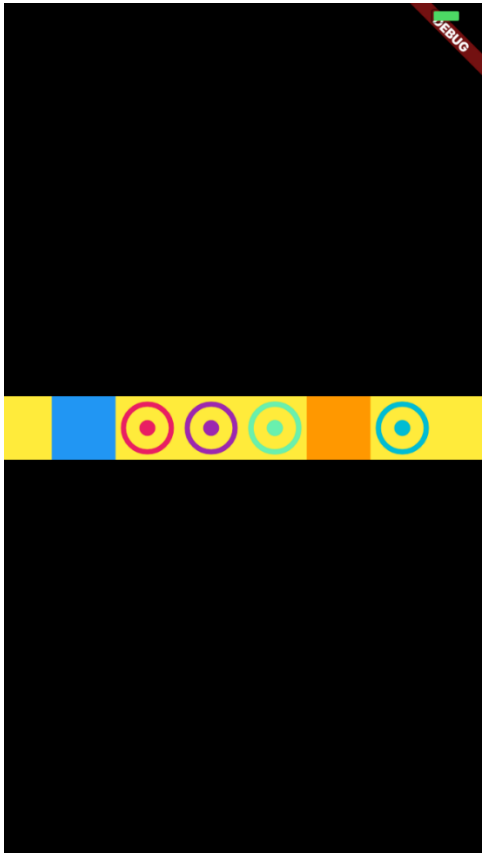
Column



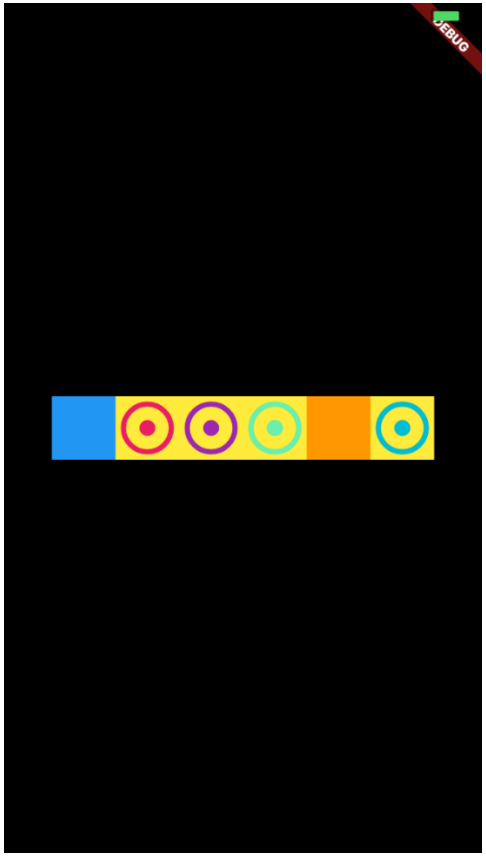
MainAxisSize Property

- The size that should be allocated to the widget on the main axis.
  - MainAxisSize.max
  - MainAxisSize.min

MainAxisSize.max



MainAxisSize.min



Any question?

References

- https://docs.flutter.dev/
- https://medium.com/jlouage/container-de5b0d3ad184
- https://medium.com/jlouage/flutter-row-column-cheat-sheet-78c38d242041
- https://medium.com/flutter-community/flutter-layout-cheat-sheet-5363348d037e

Fakultas Informatika
School of Computing
Telkom University

THANK YOU